

Práctica 3: Raft 1ª parte



**Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza**

Rubén Agustín Galdeano (820829)

Antonio González Almela (143045)

Contenido

1.- Introducción	3
2.- Resumen de requisitos	3
3.- Diseño e implementación.....	3
3.1. Estados y temporizadores	3
3.2. Elecciones (RequestVote)	4
3.3. Latidos y replicación (AppendEntries).....	5
3.4. Compromiso y aplicación a la máquina de estados	5
3.5. Concurrencia, RPC y robustez	5
4. Despliegue y ejecución	6
5. Pruebas realizadas y resultados	6
6. Limitaciones y trabajo futuro	7
7. Conclusiones	7
Anexo: Puntos de código destacables	8

1.- Introducción

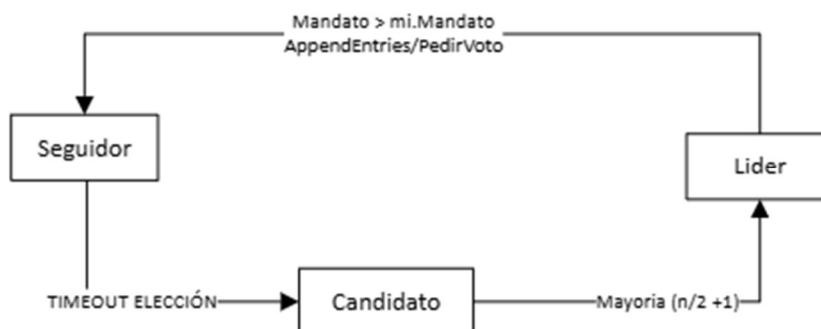
El objetivo de esta práctica es implementar los componentes esenciales del algoritmo Raft, un protocolo de consenso distribuido diseñado para ser fácil de entender y asegurar consistencia en sistemas distribuidos replicados. Los componentes que hemos integrado en esta práctica son: temporizador de elección aleatorizado, elección de líder, envío de latidos (heartbeats) mediante AppendEntries, reglas de replicación del log, compromiso por mayoría y aplicación de entradas comprometidas a la máquina de estados a través de un canal. Nuestra implementación expone un API RPC mínimo para inspección del estado y realización de operaciones, y ha sido probada con una pequeña aplicación.

2.- Resumen de requisitos

- Gestión de roles **Follower/Candidate/Leader** y temporizadores de elección.
- Petición de voto RequestVote con regla *up-to-date* (comparación LastLogTerm/LastLogIndex).
- Heartbeats periódicos AppendEntries (vacíos o con entradas), reseteando el timeout de elección.
- Replicación con chequeo de PrevLogIndex/PrevLogTerm y manejo de conflictos (retroceso de nextIndex).
- **Commit por mayoría y envío al canal** de aplicación cuando el commitIndex avanza.
- API: ObtenerEstadoNodo, SometerOperacionRaft y apagado ordenado.

3.- Diseño e implementación

3.1. Estados y temporizadores

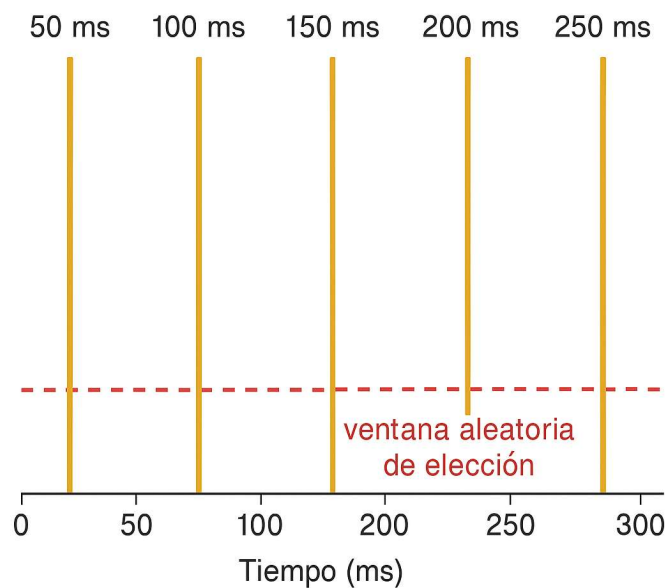


Cada réplica mantiene el estado compartido en la estructura `NodoRaft` (término, voto, log, índices volátiles y, en líder, `nextIndex/matchIndex`). Usamos un `sync.Mutex` para exclusión mutua. El temporizador de elección se **aleatoriza** en $[150, 300]$ ms mediante un generador `rand` por nodo para evitar reinicios sincronizados. Durante el arranque, se introduce un **primer retardo** (~2.5 s) que garantiza que los tests iniciales observen mandato 0 y ausencia de líder antes de la primera elección.

- Estructura con campos de rol y temporización (`role`, `timer`, `electionReset`).
- `randomizedElectionTimeout()` + `resetElectionTimerLocked()` (con la ventana inicial y posterior aleatoria).
- Bucle principal `run()` que pasa a candidato al dispararse el timeout.

Además, se incluye un **watchdog** ligero que aborta si una réplica pasa demasiado tiempo sin líder ni latidos, útil en depuración en local.

3.2. Elecciones (RequestVote)



Al vencer el temporizador, el nodo se convierte en **Candidate**, incrementa `currentTerm`, vota por sí mismo y envía `PedirVoto` al resto. Se computan votos en concurrencia y, con **mayoría**, se transiciona a **Leader**, inicializando `nextIndex` y `matchIndex`. La lógica de voto implementa la comparación *up-to-date*:

- Acepta si no ha votado o ya votó por el candidato, **y** el log del candidato está tan actualizado como el mío (LastLogTerm y, en empate, LastLogIndex).
- Si el candidato trae un término mayor, actualizamos y pasamos a follower.

Entradas relevantes: startElection(), PedirVoto(...), becomeLeaderLocked(), becomeFollowerLocked().

3.3. Latidos y replicación (AppendEntries)

El líder emite heartbeats periódicos con enviarHeartbeats(), y el seguidor **resetea** su timeout al recibir AppendEntries (aun sin entradas). Para replicar, el líder envía entradas a partir de nextIndex[peer] y requiere coincidencia en PrevLogIndex/PrevLogTerm. Ante **conflictos**, se reduce progresivamente nextIndex y se reintenta hasta lograr alineamiento.

- En seguidor: si PrevLog no coincide, se rechaza; si coincide, se trunca la parte en conflicto y se **anexan** nuevas entradas.
- Se propaga el LeaderCommit para que los seguidores adelanten su commitIndex.
- **Entradas relevantes:** AppendEntries(...), pushLogToPeer(...), enviarAppendEntries(...), enviarHeartbeats().

3.4. Compromiso y aplicación a la máquina de estados

El líder intenta avanzar commitIndex buscando el mayor N tal que **una mayoría** de réplicas tiene matchIndex $\geq$ N (y la entrada N es de su término, regla conservadora). Al avanzar commitIndex, invocamos applyCommittedEntriesLocked() que **emite** por canalAplicarOperacion todas las entradas entre lastApplied+1 y commitIndex. De este modo, la máquina de estados del servicio recibe **en orden** (indice, operacion).

- tryAdvanceCommitIndexLocked() calcula el nuevo commitIndex y **llama** a applyCommittedEntriesLocked().
- applyCommittedEntriesLocked() empaqueta y **envía** al canal fuera de sección crítica (evitando bloquear el mutex mientras se escribe en el canal).

Este punto cumple el requisito de la práctica de **aplicar operaciones comprometidas** a la máquina de estados mediante el canal provisto por NuevoNodo(...).

3.5. Concurrencia, RPC y robustez

- **RPC:** se usa el transporte rpctimeout.HostPort con métodos enviarPeticonVoto y enviarAppendEntries, con timeouts acotados. El registro RPC expone “NodoRaft” y los métodos del API de pruebas.

- **Bloqueos:** se protege el estado con Mux. Se **suelta** el mutex para realizar RPCs y se vuelve a tomar al procesar respuestas.
- **Inicialización:** el log reserva `logEntries[0]` como hueco para indexado a partir de 1; `nextIndex` se inicializa a `lastLogIndex+1` en líder.

4. Despliegue y ejecución

El binario de servidor (`cmd/srvraft/main.go`) construye la lista de nodos, crea el canal `canalAplicarOperacion`, instancia el `NodoRaft` y registra el objeto RPC. El listener TCP atiende las llamadas del conjunto de tests. Opcionalmente, un cliente de arranque puede detectar el líder y someter una operación de prueba; en paralelo, se imprime por consola cada **aplicación** recibida en el canal. *(Este main es de apoyo para pruebas manuales; el conjunto lanza réplicas con `go run` por SSH.)*

5. Pruebas realizadas y resultados

Se han utilizado los tests de integración proporcionados (arranque de 3 réplicas, sondeo de líder, fallo/reelección y acuerdos con y sin desconexiones):

- **T1 – Solo arranque y parada.** El retraso inicial del timeout evita que se elija líder prematuramente; todas reportan mandato 0 y sin líder.
- **T2 – Elegir primer líder.** Con timeouts aleatorios independientes, una réplica alcanza mayoría y pasa a líder; `enviarHeartbeats()` estabiliza el clúster.
- **T3 – Fallo y reelección.** Tras detener el líder, los seguidores inician elección; el nuevo líder emerge por mayoría. Las transiciones usan `becomeFollowerLocked` ante términos mayores.
- **T4 – Operaciones comprometidas en situación estable.** `SometerOperacionRaft` añade al log en líder, **replica** y, con mayoría, **avanza `commitIndex` y aplica** en canal. El orden es FIFO por índice.
- **T5 – Acuerdos con fallos/particiones parciales y concurrencia.**
 - Con un seguidor desconectado (mayoría 2/3), el acuerdo progresa; `pushLogToPeer` reintenta y ajusta `nextIndex` al reconectar.
 - Sin mayoría, no hay compromiso (seguridad).
 - Con **operaciones concurrentes** desde el líder, los índices avanzan monótonamente; `tryAdvanceCommitIndexLocked` asegura orden de aplicación.

En pruebas manuales, se observa la traza de **aplicaciones** recibidas en el proceso que consume `canalAplicarOperacion`, evidenciando la entrega ordenada a la máquina de estados.

6. Limitaciones y trabajo futuro

- **Persistencia:** no se han implementado almacenamiento estable ni recuperación tras fallo. Al reiniciar, se pierde `log/currentTerm/votedFor`. (*Futuro: persistir y restaurar.*)
- **Instantánea (snapshotting):** no se implementa compactación del log. (*Futuro: instalar snapshots y `InstallSnapshot RPC`.*)
- **Optimizaciones de conflictos:** el retroceso de `nextIndex` es lineal; se podría añadir pista de **términos conflictivos** para saltos más rápidos.
- **Cliente/idempotencia:** no hay deduplicación de peticiones ni redirección automática de clientes al líder (más allá del retorno del campo `EsLider/IdLider`).

7. Conclusiones

La implementación desarrollada cumple los objetivos de la Práctica 3: temporización de elecciones, **elección de líder estable**, latidos periódicos, **replicación** con chequeos de coherencia, **compromiso por mayoría** y, crucialmente, **aplicación a la máquina de estados** mediante el **canal** provisto por la infraestructura de la práctica. Las decisiones de concurrencia (bloqueo fino y liberación para RPC), el temporizador aleatorio y la regla *up-to-date* garantizan seguridad y progreso en condiciones normales y bajo fallos parciales, preparando el terreno para extender a persistencia y snapshots en prácticas posteriores.

Anexo: Puntos de código destacables

- **Temporizadores y roles:** randomizedElectionTimeout, run, runWatchdog, resetElectionTimerLocked,.
- **Elección:** startElection, PedirVoto, becomeLeaderLocked, becomeFollowerLocked.
- **Replicación/log:** AppendEntries, pushLogToPeer, enviarHeartbeats.
- **Compromiso y aplicación:** tryAdvanceCommitIndexLocked, applyCommittedEntriesLocked.
- **API:** ObtenerEstadoNodo, SometerOperacionRaft, ParaNodo.